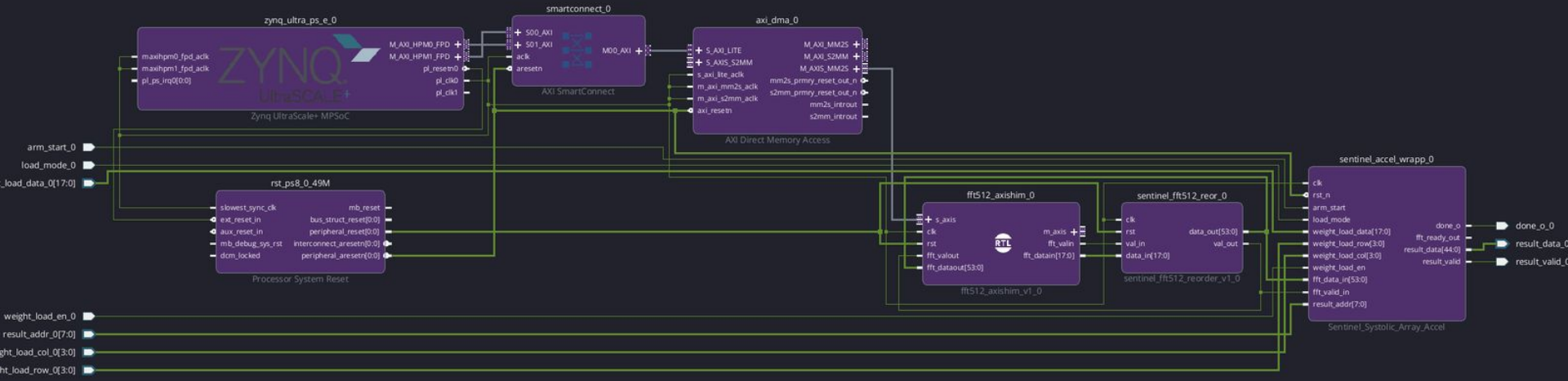
Sentinel Project: Hardware Block Diagram

Current Hardware Architecture with FFT, AXI Shim, and Systolic Array

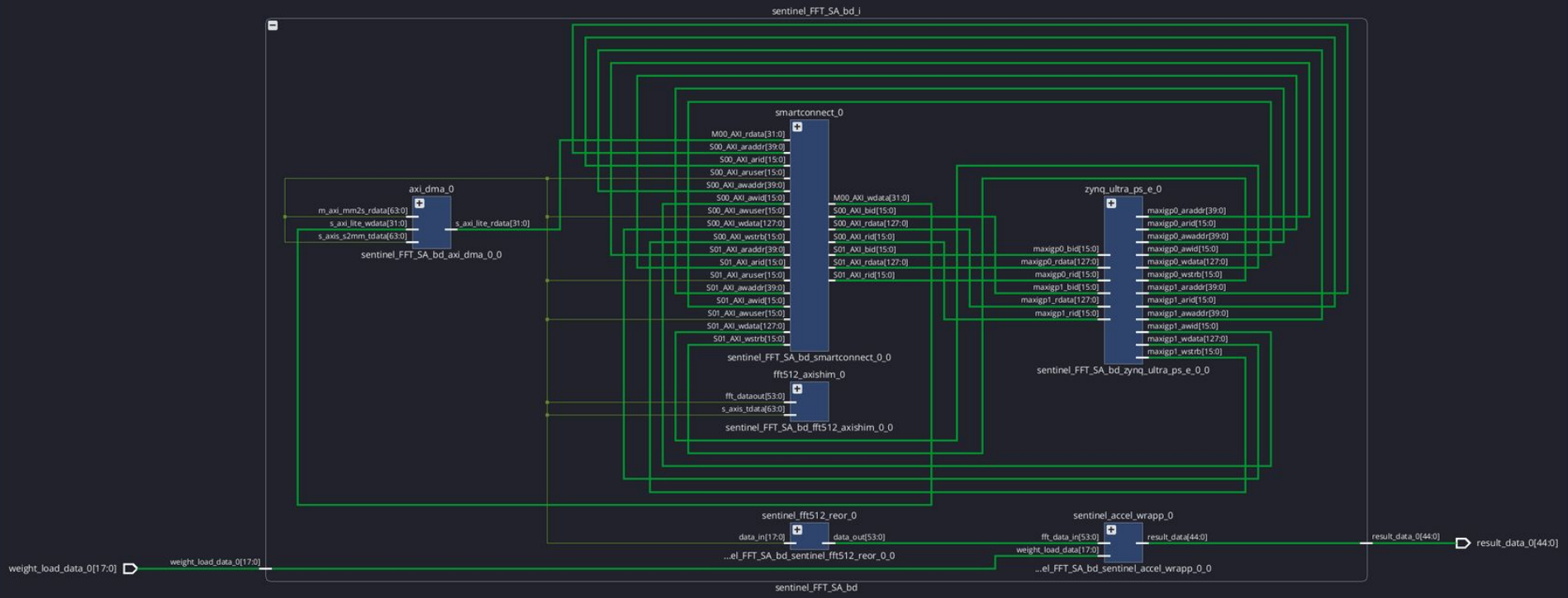
s block diagram illustrates the current Sentinel project hardware implementation on the Zynq UltraScale+ MPSoC platform. The design incorporates a Fast Fourier Transform (FFT) module for signal processing, an AXI Shim for interface adaptation, 6x16 Systolic Array architecture for parallel computation. The system utilizes AXI Direct Memory Access (DMA) for efficient data transfer between the Processing System and Programmable Logic, with the SmartConnect IP managing high-performa l interconnects.



Sentinel Project: Data Flow Design

Vivado Data Flow Schematic from IP Integrator

This data flow schematic, generated from Xilinx Vivado's IP Integrator, shows the complete data path architecture of the Sentinel FFT SA system. The diagram illustrates how data flows through the AXI DMA controller, SmartConnect interconnect, FFT512 AXI Shim, and the Sentinel Systolic Array accelerator. The design demonstrates the integration of memory-mapped AXI interfaces with stream-based data processing for high-throughput signal analysis.



Sentinel Project: Implementation Reports

Power Analysis, Resource Utilization, and Timing Summary

The following reports summarize the post-implementation analysis of the Sentinel project on the Zynq UltraScale+ platform. The design achieves timing closure with a Worst Negative Slack (WNS) of 0.953 ns. Power consumption is optimized at 3.012 W total on-chip power, with 90% attributed to dynamic power. Resource utilization shows efficient use of DSP slices (26.28%) and BRAM (90.28%).

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**3.012 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**32.0°C**
Thermal Margin	53.0°C (22.6 W)
Ambient Temperature	25.0 °C
Effective ϑJA	2.3°C/W
Power supplied to off-chip devices	0 W
Confidence level	Low

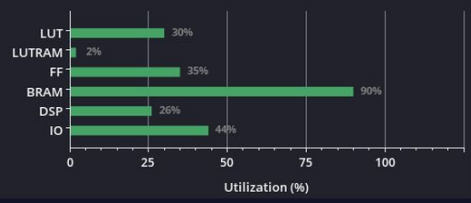
Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power

Power Analysis Summary

Summary

Resource	Utilization	Available	Utilization %
LUT	34582	117120	29.53
LUTRAM	1257	57600	2.18
FF	81837	234240	34.94
BRAM	130	144	90.28
DSP	328	1248	26.28
IO	84	189	44.44



Resource Utilization Summary

Hierarchy

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	F8 Muxes (29280)	Block RAM Tile (144)	DSPs (1248)	Bonded IOB (189)	BUFG_PS (96)	PS8 (1)
N sentinel_FFT_SA_bd_wrapper	36459	81990	447	8895	4221	130	328	84	2	1
I sentinel_FFT_SA_bd_i (sentinel_FFT_SA_bd)	36459	81990	447	8895	4221	130	328	0	2	1
axi_dma_0 (sentinel_FFT_SA_bd_axi_dma_	1035	1188	15	5	0	2	0	0	0	0
sentinel_accel_wrapp_0 (sentinel_FFT_SA_	10097	21184	4	1530	765	128	256	0	0	0
inst (sentinel_accel_wrapper)	10097	21184	4	1530	765	128	256	0	0	0
fe_inst (feature_extractor)	425	32	4	0	0	0	0	0	0	0
sa_inst (systolic_array_16x16)	9672	21152	0	1530	765	128	256	0	0	0
sentinel_fft512_reor_0 (sentinel_FFT_SA_b	22781	57220	410	7360	3456	0	72	0	0	0
inst (FFTS12_Reorder_IP)	22781	57220	410	7360	3456	0	72	0	0	0
fft512 (FFTS12)	6611	1699	410	16	0	0	72	0	0	0
sdff1 (SDF_mod)	1022	395	42	16	0	0	8	0	0	0
bf (Butterfly_92)	296	0	42	0	0	0	8	0	0	0
shift_reg (Shift_Reg)	569	364	0	0	0	0	0	0	0	0
sdf2 (SDF_mod__parameterized0)	866	269	46	0	0	0	8	0	0	0
bf (Butterfly_91)	294	0	42	0	0	0	8	0	0	0
bf_fsm (Butterfly_FSM__param	79	33	0	0	0	0	0	0	0	0
shift_reg (Shift_Reg__parameter	459	236	0	0	0	0	0	0	0	0
sdf3 (SDF_mod__parameterized1)	706	204	46	0	0	0	8	0	0	0
sdf4 (SDF_mod__parameterized2)	638	171	46	0	0	0	8	0	0	0
sdf5 (SDF_mod__parameterized3)	594	154	46	0	0	0	8	0	0	0
sdf6 (SDF_mod__parameterized4)	574	148	46	0	0	0	8	0	0	0
sdf7 (SDF_mod__parameterized5)	568	142	46	0	0	0	8	0	0	0
sdf8 (SDF_mod__parameterized6)	482	137	46	0	0	0	8	0	0	0
sdf9 (SDF_mod__parameterized7)	1177	79	46	0	0	0	8	0	0	0
reorder_buffer (Reorder_Buffer)	16170	55521	0	7344	3456	0	0	0	0	0
smartconnect_0 (sentinel_FFT_SA_bd_sma	2257	2353	18	0	0	0	0	0	0	0
zynq_ultra_ps_e_0 (sentinel_FFT_SA_bd_zy	264	0	0	0	0	0	0	0	2	1

Detailed Hierarchy Utilization

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	0.953 ns	Worst Hold Slack (WHS)	0.012 ns	Worst Pulse Width Slack (WPWS)	8.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	202003	Total Number of Endpoints	202003	Total Number of Endpoints	84132

All user specified timing constraints are met.

Design Timing Summary - All Constraints Met

PE Cell Module - SystemVerilog Implementation

Processing Element for Systolic Array with BRAM-based Weight Storage

```
`timescale 1ns / 1ps


module pe_cell #
(
    parameter row = 0,
    parameter column = 0,
    parameter K = 512
)

(
    input logic clk, rst,

    // This is mainly for flowing between PE cells
    input logic signed [PE_INP_WIDTH-1:0]  activation_i,        // input activation
    output logic signed [PE_INP_WIDTH-1:0] activation_o,        // output activation

    // Variables for weight loading
    input logic signed [PE_INP_WIDTH-1:0]  weight_load_data,
    input logic                            weight_load_en,
    input logic                            load_mode,

    // Control variables
    input logic  enable_i,                                      // enable input for starting computation
    output logic enable_o,                                      // enable output for mainly just carrying on the enable to the next neighboring tile

    // Outputting results
    output logic signed [PE_ACCUM_WIDTH-1:0] accum_o,           // accumulation variable from weight and activation variable multiplications (and additions onto this one)
    output logic                             valid_o            // valid signal for indicating when accumulation is finished (active-high)
);
    // General variables for clocking new values (while outputs receive these combinationally)
    logic                              valid_reg;
    logic                              enable_reg;
    logic signed [PE_INP_WIDTH-1:0]    activation_reg;
    logic signed [PE_ACCUM_WIDTH-1:0]  accum_reg;

    // BRAM for holding weights locally (this will be updated/loaded by the arm core initially) (this addition caused a bit more complexity)
    (* ram_style = "block" *)
    logic signed [PE_INP_WIDTH-1:0] weight_mem [0:K-1];
    logic signed [PE_INP_WIDTH-1:0] weight_data_reg;            // variable for registering weight from BRAM (so it actually infers BRAM instead of distributed RAM)

    logic [$clog2(K)-1:0]           weight_addr;                // counter for loading new weights

    logic [$clog2(K):0]             compute_addr;               // counter for checking where we are at with computation for the systolic array for this specific pe_tile
                                                                // needs the extra bit to hold values >= K (512) for the termination check


    // Flag to track if we've finished computation (stops the accumulator from going undefined from using undefined inputs)
    logic                           done_flag;

    // Flag to track BRAM prefetch (first cycle loads weight[0] into reg without accumulating)
    logic                           prefetch_done;

    initial
    begin
        valid_reg = 0;
        enable_reg = 0;
        activation_reg = 0;
        accum_reg = 0;
        done_flag = 0;
        prefetch_done = 0;
    end

    always_ff @(posedge clk)
    begin
        if (rst)
        begin
            enable_reg <=       0;
            activation_reg <=   0;
            accum_reg <=        0;
            compute_addr <=     0;
            weight_addr <=      0;
            valid_reg <=        0;
            done_flag <=        0;
            prefetch_done <=    0;
        end

        else if (load_mode)
        begin
            // LOAD PHASE: where the weights of this specific pe_tile are being loaded into the tile's BRAM
            if(weight_load_en)
            begin
                weight_mem[weight_addr] <=  weight_load_data;
                weight_addr <= (weight_addr >= K-1) ? 0 : weight_addr + 1; //wraparound
            end

            enable_reg <= 0; // Make sure enable_reg isn't on when loading (otherwise, obviously problems)
            done_flag <= 0;  // Reset done flag during load
            prefetch_done <= 0; // Reset prefetch so next compute phase starts fresh
        end

        else
        begin
            // BRAM READ: Always read current address at start of cycle (this forces BRAM inference since it's registered)
            if ((enable_i || enable_reg) && !done_flag)
                weight_data_reg <= weight_mem[compute_addr];

            if ((enable_i || enable_reg) && !done_flag)
            begin
                if (enable_i) enable_reg <= 1;

                // PREFETCH CYCLE: First cycle loads weight[0] into reg, increment addr for next read
                // otherwise there would be a 1-cycle read latency of BRAM (which would mess up what activation multiplies with what weight)
                if (!prefetch_done)
                begin
                    prefetch_done <=    1'b1;
                    compute_addr <=     compute_addr + 1;  // 0 -> 1 (next cycle reads weight[1])
                    // No accumulation this cycle (just loading weight[0] into reg)
                end

                // FINAL MAC: When addr reaches 512, do last MAC with weight[511] in reg
                else if (compute_addr >= K)
                begin
                    activation_reg <= activation_i;      // Capture final activation for pipeline
                    accum_reg <= accum_reg + (activation_i * weight_data_reg);  // Use current activation, not reg
                    valid_reg <=    1;
                    enable_reg <=   0;
                    done_flag <=    1;
                    prefetch_done <= 0;
                    compute_addr <=  0;
                end

                // NORMAL MAC: Cycles 1-511 (addr 1-511)
                else
                begin
                    activation_reg <=   activation_i;    // Pipeline for next PE
                    accum_reg <=        accum_reg + (activation_i * weight_data_reg);  // Use current activation
                    compute_addr <=     compute_addr + 1;  // Increment for next weight read
                end
            end
        end
    end

    assign valid_o =        valid_reg;
    assign enable_o =       enable_reg;
    assign activation_o =   activation_reg;
    assign accum_o =        accum_reg;                          // Now holds stable value after completion

endmodule
```